



האקתון VLSI

פיתוח מאיץ חישובי על מעבדי RISC-V



חסות פלטינום



NVIDIA





חסות פרימיום

intel®





חסות פרימיום





חסות פרימיום

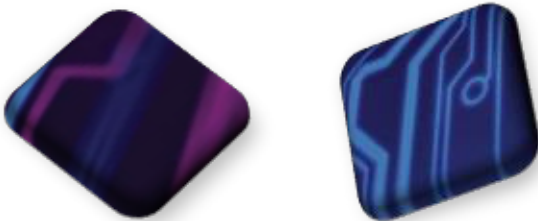
cā d e n c e[®]





חסות סילבר

toganet»orks
a HUAWEI company





חסות סילבר

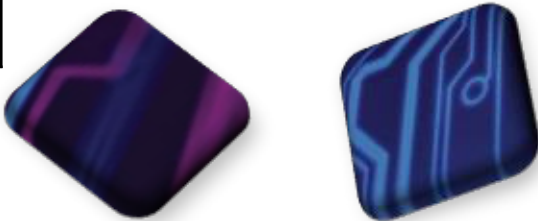
Google





לו"ז האקתון- יום ראשון 21.06

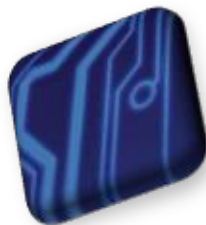
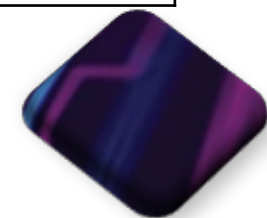
בניין ברנדמן ב', כיתה 101	רישום	8:00 – 09:00
בניין ברנדמן ב', כיתה 101	הצגה חגיגית של האתגר- פרדי	09:00 – 09:45
		09:45-10:00
בניין ברנדמן ב', כיתות 101-102	תחילת עבודה	9:45-13:00
מרחב הקומה	כיבוד	13:00-14:00
בניין ברנדמן ב', כיתות 101-102	זמן עבודה, משמרת מנטורים	14:00-16:00
	זמן עבודה, משמרת מנטורים	16:00-18:00
מרחב הקומה	כיבוד	18:00-19:00
בניין ברנדמן ב', כיתות 101-102	זמן עבודה, משמרת מנטורים	19:00- 21:30
מרחב הקומה	Happy hour	21:30-23:00





לו"ז האקתון- יום שני 22.06

מודל	העלאת תוצאות ההאקתון למודל	8:30
מבואה ראשית, קומה 2	כיבוד	8:00-9:00
בניין ברנדמן ב', כיתות 101-103	הכרזה על הצוותים שיעלו לגמר	9:00-09:05
בניין ברנדמן ב', כיתות 101	שלב הגמר: הצוותים הנבחרים עולים בזה אחר זה ומציגים 2-3 דק'	09:05-10:05
	בדיקת תוצאות השיפוט, הפסקה	10:05-10:15
בניין ברנדמן ב', כיתה 101	סיכום והכרזה על המנצחים חלוקת GIVEAWAYS	10:15-10:30



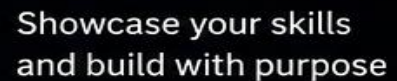
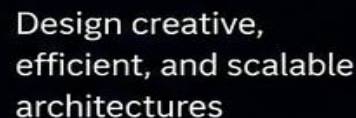
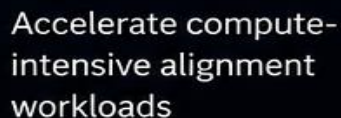
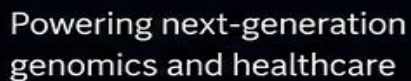
Build the future of genomics on FPGA



0	0	0	0	0	0	0	0	0
2	1	1	3	6	5	7	9	2
4	3	3	2	2	4	6	3	3
6	5	5	4	3	5	5	6	2
8	7	7	6	5	5	4	6	1
10	9	9	8	7	7	6	3	3
12	11	10	9	8	8	8	12	3

Best local alignment

Smith-Waterman with Affine Gap Penalty



DNA Basics

- DNA is represented by four nucleotides:
 - A = Adenine
 - C = Cytosine
 - G = Guanine
 - T = Thymine
- Example DNA sequence:
 - ACGTACGT
- Why Alignment Matters?
 - Compare DNA sequences to find similarity.
 - Applications: sequencing, disease analysis, ancestry, genome assembly.

Input Sequences

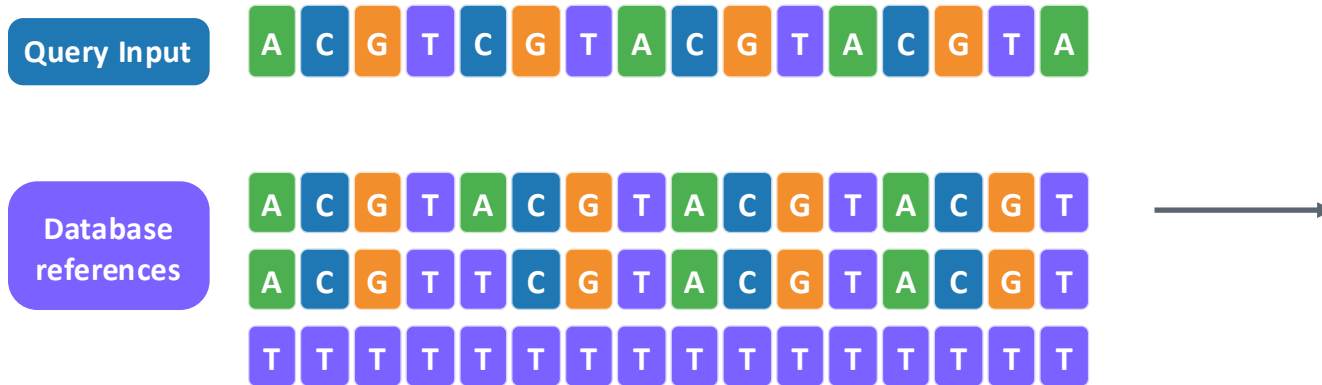
- Input contains ONLY A,C,G,T.
- Reference: ACGTACGT
- Query: ACGTCGT
- No '-' characters exist in the input.

What is a Gap?

- The algorithm may introduce gaps to improve alignment.
- Reference: ACGTACGT
- Query: ACGT-CGT
- '-' is created by the algorithm, not present in the input.

What will build at the hackathon?

Given a query DNA sequence and many reference sequences, compute the best local **alignment score** for each reference.



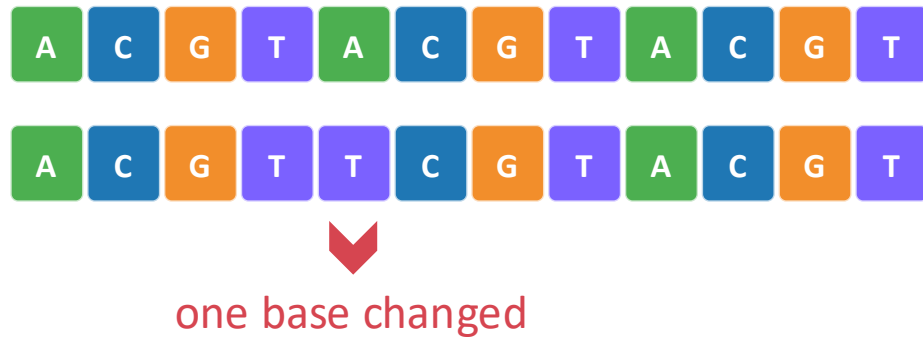
Hardware accelerator returns scores

Ref	Score
0	26
1	26
2	2

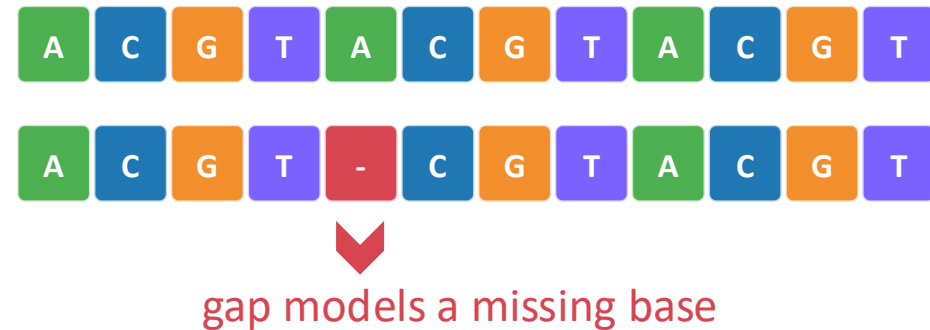
Why alignment is not exact string matching

Exact comparison fails when a read has a mutation or a missing base.

Example 1: mismatch



Example 2: insertion/deletion



Alignment finds the highest-scoring way to place matches, mismatches and gaps between two sequences.

Scoring model

Event	Score	Meaning
Match	+2	same DNA base
Mismatch	-1	different base
Gap open	-4	start insertion/deletion
Gap extend	-1	continue an existing gap

Why two gap scores?



one long gap: $-4 -1 -1 -1 = -7$



many new gaps: $-4 -4 -4 -4 = -16$

Affine gaps are more realistic: opening a gap is expensive, extending it is cheaper.

Basic Smith-Waterman Algorithm:

Start with a dynamic-programming matrix

Rows are query bases.

Columns are reference bases.

Each cell stores the best local alignment ending at that position.

	A	C	G	T
A				
G				
T				



Reference: A C G T

Query: A G T

Each cell asks:

“What is the best score if the alignment ends here?”

Smith-Waterman is local alignment: any cell may reset to zero and start a new alignment.



The basic cell update

	A	C	G	T
A				
G				
T				

To compute H, use only its neighbors:

↖ diagonal ↑ up ← left

Basic recurrence

$$H(i,j) = \max(0, \text{diag} + \text{score}, \text{up} + \text{GAP}, \text{left} + \text{GAP})$$

```
1 int diag = H[i-1][j-1] + score;
2 int up   = H[i-1][j]   + GAP;
3 int left = H[i][j-1]   + GAP;
4 H[i][j] = max4(0, diag, up, left);
```

This version treats every gap character the same.
The next step is affine gaps.

Affine-gap Smith-Waterman Algorithm

To distinguish “gap open” from “gap extend”, one matrix is no longer enough. We track three states for each cell.

M Match / Mismatch
character aligned to character

I Insertion
vertical gap is active

D Deletion
horizontal gap is active

Each Dynamic Programming (DP) cell updates M, I, and D.

Affine-gap recurrences

Insertion state

$$I(i,j) = \max(M(i-1,j) + \text{GAP_OPEN}, I(i-1,j) + \text{GAP_EXT})$$

Deletion state

$$D(i,j) = \max(M(i,j-1) + \text{GAP_OPEN}, D(i,j-1) + \text{GAP_EXT})$$

Match state

$$M(i,j) = \max(0, M(i-1,j-1) + s(i,j), I(i,j), D(i,j))$$

The best score is the maximum value seen in M over the whole matrix.

C code: constants and storage

Step 1

```
1 #define MATCH      2
2 #define MISMATCH  -1
3 #define GAP_OPEN   -4
4 #define GAP_EXT    -1
5
6 #define MAX_LEN    32
7 #define NEG_INF    (-1000000)
8
9 static int M[MAX_LEN][MAX_LEN];
10 static int I[MAX_LEN][MAX_LEN];
11 static int D[MAX_LEN][MAX_LEN];
```

M, I, and D are the three DP states.
NEG_INF prevents impossible gap states
from winning before they are valid.

M matrix



I matrix



D matrix



```
1 int rows = strlen(query) + 1;
2 int cols = strlen(ref) + 1;
3 int best_score = 0;
4
5 for (int i = 0; i < rows; i++) {
6     for (int j = 0; j < cols; j++) {
7         M[i][j] = 0;
8         I[i][j] = NEG_INF;
9         D[i][j] = NEG_INF;
10    }
11 }
```

Why initialize this way?

- M starts at 0 because local alignment may start anywhere.
- I and D start impossible because a gap cannot exist before an alignment begins.
- best_score tracks the strongest local match seen so far.

C code: inner loop

Step 3

```
1 for (int i = 1; i < rows; i++) {
2     for (int j = 1; j < cols; j++) {
3         int s = (query[i-1] == ref[j-1])
4             ? MATCH : MISMATCH;
5
6         I[i][j] = max2(M[i-1][j] + GAP_OPEN,
7                       I[i-1][j] + GAP_EXT);
8
9         D[i][j] = max2(M[i][j-1] + GAP_OPEN,
10                      D[i][j-1] + GAP_EXT);
11
12        M[i][j] = max4(0,
13                      M[i-1][j-1] + s,
14                      I[i][j],
15                      D[i][j]);
16
17        if (M[i][j] > best_score)
18            best_score = M[i][j];
19    }
20 }
```

Per cell

1. Choose match/mismatch score
2. Update insertion state
3. Update deletion state
4. Update match state
5. Update global best score

Walkthrough example: one cell update

Assume a cell where $\text{query}[i-1]$ equals $\text{ref}[j-1]$, so $s = +2$.

Input value	Number
$M(i-1, j-1)$	10
$M(i-1, j)$	7
$I(i-1, j)$	3
$M(i, j-1)$	8
$D(i, j-1)$	6

```
1 I = max(7 + -4, 3 + -1) = 3
2 D = max(8 + -4, 6 + -1) = 5
3 M = max(0, 10 + 2, 3, 5) = 12
```

**Result: the best alignment ending here has score
12.**

Example alignment behavior

Two possible alignments with similar characters but different gap structure:

One long gap



$$\text{Gap penalty} = -4 -1 -1 -1 = -7$$

Preferred when the biology suggests one indel event.

Many short gaps



$$\text{Gap penalty} = -4 -4 -4 -4 = -16$$

Penalized more heavily: multiple gap-open events.

Hackathon input data

```
1 const char *query =  
2     "ACGTCGTACGTACGTA";  
3  
4 const char *references[] = {  
5     "ACGTACGTACGTACGT",  
6     "ACGTTCGTACGTACGT",  
7     "ACGTACGGACGTACGT",  
8     "TTTTTTTTTTTTTTTT",  
9     "ACGTACGTTCGTACGT",  
10    "ACGTACGTACGTACGA",  
11    "ACGTTTGTACGTACGT",  
12    "ACGTACGTGCGTACGT"  
13 };
```

Expected output for starter dataset

Reference	Expected score
0	26
1	26
2	23
3	2
4	23
5	24
6	23
7	23

```
1 for (int i = 0; i < num_refs; i++) {  
2     int score = smith_waterman_affine(  
3         references[i], query);  
4  
5     printf("Reference %d score = %d\n",  
6         i, score);  
7 }
```

Rule: an accelerator must produce the same scores as the C reference before it can be ranked.

Complete C reference: helpers and function header

Code appendix 1/3

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MATCH      2
5 #define MISMATCH  -1
6 #define GAP_OPEN   -4
7 #define GAP_EXT    -1
8 #define MAX_LEN    32
9 #define NEG_INF     (-1000000)
10
11 static int M[MAX_LEN][MAX_LEN];
12 static int I[MAX_LEN][MAX_LEN];
13 static int D[MAX_LEN][MAX_LEN];
14
15 static inline int max2(int a, int b) {
16     return (a > b) ? a : b;
17 }
18 static inline int max4(int a, int b, int c, int d) {
19     int m = a;
20     if (b > m) m = b;
21     if (c > m) m = c;
22     if (d > m) m = d;
23     return m;
24 }
```

Complete C reference: affine-gap kernel

Code appendix 2/3

```
1 int smith_waterman_affine(const char *ref,
2                           const char *query)
3 {
4     int rows = strlen(query) + 1;
5     int cols = strlen(ref) + 1;
6     int best_score = 0;
7
8     for (int i = 0; i < rows; i++)
9         for (int j = 0; j < cols; j++) {
10             M[i][j] = 0;
11             I[i][j] = NEG_INF;
12             D[i][j] = NEG_INF;
13         }
14
15     for (int i = 1; i < rows; i++) {
16         for (int j = 1; j < cols; j++) {
17             int s = (query[i-1] == ref[j-1])
18                 ? MATCH : MISMATCH;
19
20             I[i][j] = max2(M[i-1][j] + GAP_OPEN,
21                           I[i-1][j] + GAP_EXT);
22             D[i][j] = max2(M[i][j-1] + GAP_OPEN,
23                           D[i][j-1] + GAP_EXT);
```

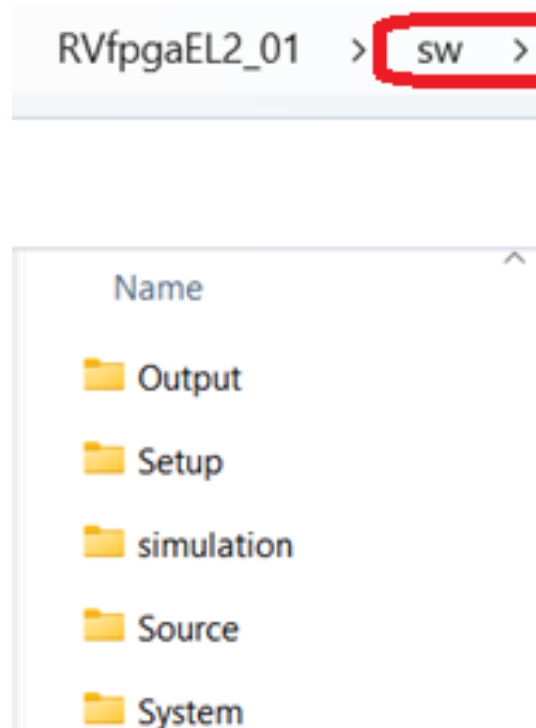
Complete C reference: finish kernel + main

Code appendix 3/3

```
1      M[i][j] = max4(0,
2                      M[i-1][j-1] + s,
3                      I[i][j],
4                      D[i][j]);
5
6      if (M[i][j] > best_score)
7          best_score = M[i][j];
8  }
9  }
10 return best_score;
11 }
12
13 int main(void)
14 {
15     const char *query = "ACGTCGTACGTACGTA";
16     const char *references[] = {
17         "ACGTACGTACGTACGT", "ACGTTTCGTACGTACGT",
18         "ACGTACGGACGTACGT", "TTTTTTTTTTTTTTTT",
19         "ACGTACGTTTCGTACGT", "ACGTACGTACGTACGA",
20         "ACGTTTGTACGTACGT", "ACGTACGTGCGTACGT"
21     };
22     int num_refs = sizeof(references) / sizeof(references[0]);
23     for (int i = 0; i < num_refs; i++)
24         printf("Reference %d score = %d\n", i,
25               smith_waterman_affine(references[i], query));
26     return 0;
27 }
```

קומפילציה והרצה

- את התכנית יש לקמפל ולהריץ בסביבת ה- Segger ES כפי שעשינו בתרגיל ההכנה.



- את הפרויקט של האתגר יש להוריד מהמודל ולעשות unzip תחת התיקיה sw. יש להשתמש בקובץ הפרויקט שמסופק בתיקייה זו ולא בכל קובץ פרויקט אחר.

פלט התכנית

- לתכנית יש פלט מובנה – אין לשנות או למחוק פלט זה.
- הפלט מחשב את
- זמני התחלה, סיום וזמן ריצה כולל
- שימו לב כי חישובי הזמנים הם ביחידות של זמן המחזור של המעבד
- חישובי הזמנים מתבססים על מוני הזמן המובנים של המעבד

פלט התוכנית

זמן ביצוע

```
Debug Terminal
Cycles = 1157276
Reference 0 score = 26
Reference 1 score = 26
Reference 2 score = 23
Reference 3 score = 2
Reference 4 score = 23
Reference 5 score = 24
Reference 6 score = 23
Reference 7 score = 23
```

DNA Scoring Results

קריטריונים לשיפוט

הקריטריונים הינם בסדר העדיפות המפורט להלן:

1. זמן ביצוע של כלל משימת החישוב.

2. שימוש מינימלי ככל שניתן בחומרת ה-FPGA

Lookup Tables (LUTs) -

Muxes -

Registers -

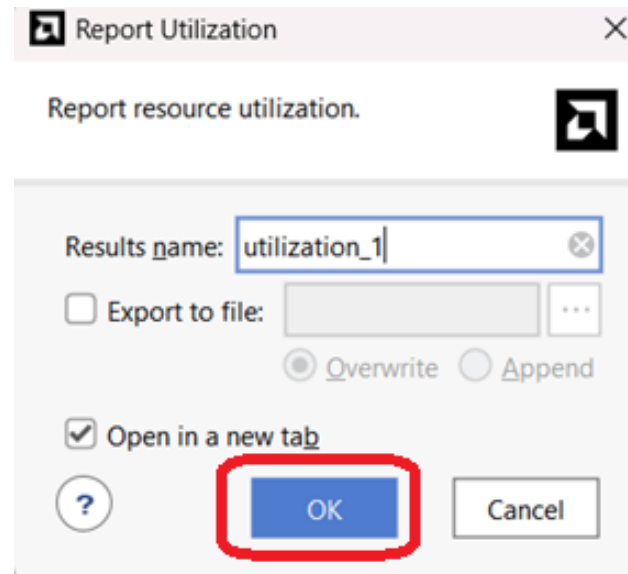
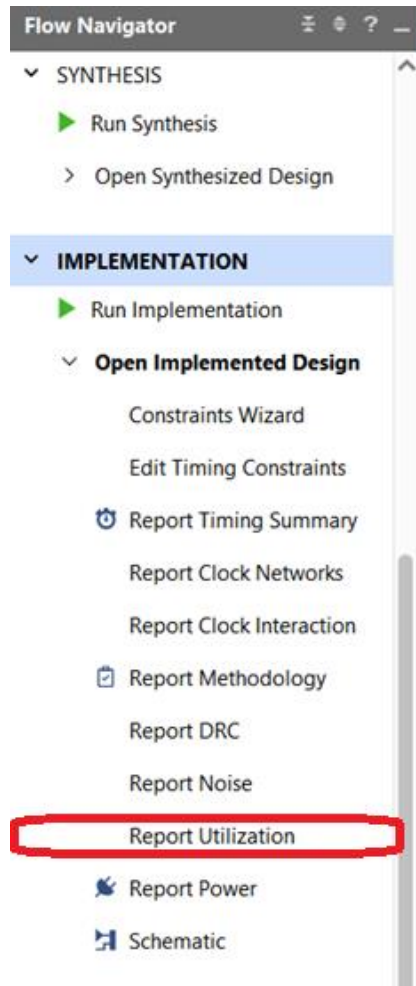
Block RAM -

DSPs -

3. חדשנות, יצירתיות ומקוריות על פי החלטת ועדת השופטים

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	DSPs (240)
rvfpganexys	36025	16014	834	292	10485	34421	1604	55.5	4
clk_gen (clk_gen_nexys)	1	2	0	0	1	1	0	0	0
dbg_hub (dbg_hub)	488	741	0	0	220	464	24	0	0
dmi_wrapper (dmi_wrapper)	62	98	0	0	34	62	0	0	0
ram (axi_ram)	109	120	0	0	47	109	0	32	0
swervolf (veerwolf_core)	35259	15001	834	292	10221	33679	1580	22.5	4
vga_mem (axi_mem)	100	36	0	0	33	100	0	1	0

יצירת דו"ח שימוש במשאבי חומרה



כללים

- אין לשנות את החישוב המתמטי של האלגוריתם או להחליף את תוצאת החישוב של האלגוריתם בקבועים.
- תוצאות האלגוריתם שתקבלו חייבות להיות תואמות לתכנית המקורית.
- אין לבצע שינוי בתדר המערכת. על תהליך הקומפילציה (סינתזה – place-and-route) להסתיים עם 0 הפרות של תזמונים
- הפתרון חייב להיות מקורי – אין להעתיק פתרונות
- אין לשנות את בסיס הנתונים
- אין לשנות את סביבת הפלט
- על הקלט והפלט של התכנית להתבצע על מעבד ה- RISC-V
- אין למחוק או לבטל את מעבד ה- RISC-V בפתרון
- כל הקלט שמור בזיכרון ה- TCM של המעבד במערך. אין להחליף את הקלט בקבועים.
- אין למחוק או לשנות את הפונקציות: `pspMachinePerfMonitorEnableAll`, `pspMachinePerfCounterSet` ו-`pspMachinePerfCounterGet`

בכל שאלה או ספק יש לפנות לסגל הקורס

שימוש בכלי בינה מלאכותית יוצרת (Generative AI)

• השימוש בכלי בינה מלאכותית יוצרת, כגון ChatGPT, Claude, GitHub Copilot, Gemini וכלים דומים, מותר במהלך ההאקתון. עם זאת, מטרת ההאקתון היא לבחון יכולות הנדסה, יצירתיות, תכנון ארכיטקטוני ואופטימיזציה. לכן חלות המגבלות הבאות:

1. האחריות המלאה על הפתרון היא של המשתתפים. כל צוות אחראי לנכונות, לאמינות ולביצועי הפתרון שהוגש, גם אם נעשה שימוש בכלי AI במהלך הפיתוח.
2. מותר להשתמש ב-AI לצורכי למידה, ייעוץ וסיוע טכני, לרבות: הבנת האלגוריתם, למידת מושגים בארכיטקטורת מחשבים, RISC-V-IFPGA, יצירת קטעי קוד חלקיים, איתור ותיקון תקלות Debug, כתיבת תיעוד ודוחות, סיעור מוחות וגיבוש רעיונות.
3. אסור להגיש פתרון מלא שנוצר באופן אוטומטי על ידי כלי AI ללא תרומה הנדסית משמעותית של הצוות. הארכיטקטורה הכוללת, אסטרטגיית ההאצה, החלטות התכנון והאינטגרציה חייבות להיות פרי עבודת הצוות.
4. **מותר לשלב קוד שנוצר באמצעות AI, ובלבד שהצוות מבין אותו באופן מלא ויכול להסביר את פעולתו.**
5. על כל צוות לדווח על השימוש ב-AI במסגרת מצגת הסיכום, לרבות: אילו כלים שימשו את הצוות, לאילו מטרות נעשה בהם שימוש, התרומה המשוערת שלהם לפתרון.
6. חל איסור להעביר בין צוותים פתרונות, קוד, תוצרים או הנחיות (Prompts) שנוצרו באמצעות AI.
7. ועדת השיפוט רשאית לבקש מכל צוות להסביר כל רכיב בפתרון.
8. **השימוש ב-AI אינו מהווה קריטריון שיפוט בפני עצמו.** ההערכה תתבסס על ביצועי המערכת, ניצול המשאבים, איכות התכנון, החדשנות והבנת הפתרון.



הגשת התוצאות

יש להעלות את מצגת הסיכום לתיבת ההגשה
בפורמט שקיים במודל

יש להגיש את הדוח המסכם לתיבת ההגשה
לא יאוחר מה 1.7.2026 על פי הפורמט
במודל

הגשת התוצאות תעשה בתיבת ההגשה
במודל

יש למלא את טופס תוצאות ההאקתון

תוצאות מספריות יש למלא בערכים
דצימליים

מילוי שאלון ההשתתפות בהאקתון הוא חובה

תיבת הגשה למצגת הסופית

תיבת הגשה לדוח מסכם

פורמט מצגת סופית

פורמט דוח מסכם

הגשת התוצאות

שאלון השתתפות בהאקתון

מיקום 1 [תשובה מספרית] (זמן ביצוע)

מהו זמן ביצוע של כלל משימת החישוב ביחידות של מחזורי
שעון?

מיקום 2 [שאלה פתוחה] (משאבי חומרת ה-FPGA)

העתיקו את פירוט טבלת השימוש במשאבים על פי הפורמט הבא:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	DSPs (240)
rvfpganexys	36025	16014	834	292	10485	34421	1604	55.5	4
clk_gen (clk_gen_nexys)	1	2	0	0	1	1	0	0	0
dbg_hub (dbg_hub)	488	741	0	0	220	464	24	0	0
dmi_wrapper (dmi_wrapper)	62	98	0	0	34	62	0	0	0
ram (axi_ram)	109	120	0	0	47	109	0	32	0
swervolf (veerwolf_core)	35259	15001	834	292	10221	33679	1580	22.5	4
vga_mem (axi_mem)	100	36	0	0	33	100	0	1	0

מצגת מסכמת

Solution Descriptions and Principles of Operation – 2 slides

- General idea
 - Short description of your idea
- Hardware acceleration
 - Indicate which parts of the algorithm have been accelerated in hardware
- Software computation
 - Describe which parts of the algorithm are processed in software on the RISCv core.



Microarchitectural and Design Solution – 2 slides

- A scheme describing the solution microarchitecture



Results Summary

Metric	Results
Total latency of the full workload	

Hardware resources:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	DSPs (240)
rvfpganexys	36025	16014	834	292	10485	34421	1604	55.5	4



Output Run Screen Shot



שאלון השתתפות

שאלון השתתפות בהאקתון

1 ?What is your current year of study

1st Year ☐

2nd Year ☐

3rd Year ☐

4th Year ☐

Other! ☐

2 ?What is your area of specialization or major

Electrical Engineering and Applied Physics ☐

Computer Science ☐

Computer Engineering ☐

Other! ☐

3 ?Have you worked on projects related to VLSI, hardware design, or RISC-V processors before this hackathon

כן ☐ לא ☐

4 ?What is your level of familiarity with hardware/software co-design

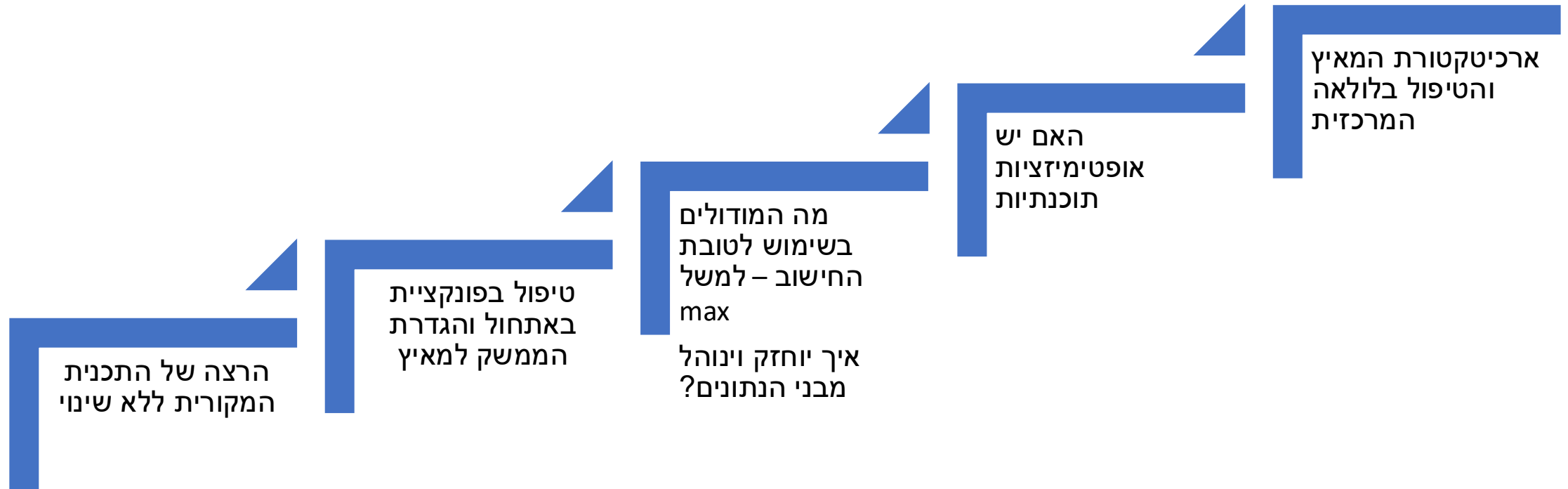
- למילוי עד ל 1.7.2026 –
חובה לשם קבלת ציון עובר
- מספר שאלות קצרות
שיסייעו מחקרית לטובת
PBL ולשיפור ההאקתונים
הבאים.



האוניברסיטה העברית בירושלים
THE HEBREW UNIVERSITY OF JERUSALEM

HUJI
Applied
Physics

טיפים – דוגמא להתקדמות בשלבים



דגשים וטיפים נוספים

- תכנון לפני שניגשים למשימה - הכינו תכנית עבודה המורכבת ממספר שלבים – מומלץ להתקדם בצורה מדורגת
- אל תרוצו מייד לפתרון – הקדישו זמן בתחילת הדרך לגבש אסטרטגיה פתרון מוסכמת בצוות
- חלוקת עבודה אפקטיבית בין השותפים לצוות
- ביצוע profiling של הקוד - חוק Amdahl – "האץ את השכיח"
- נצלו ככל שניתן את המאיץ שבניתם במשימת ההכנה לטובת המשימה הנוכחית
- בצעו checkpoints
- גיבויים - שימרו גרסאות ביניים של הקוד על מנת שתוכלו תמיד לחזור אחורה במידה ומשהו משתבש.
- הסתייעות במנטורים ובסגל
- חשיבה מערכתית, חומרה ותכנה שפועלים יחד
- העזרו בסביבה של ה unit level לבדיקת ה design



בהצלחה !!!